POO COM C#

Pedro Henrique Ribeiro Dias

Inatel

INTRODUÇÃO

Enquanto o C++ exige controle manual de memória, o C# automatiza essa tarefa com o Garbage Collector, além de apresentar uma sintaxe mais limpa e simplificada, especialmente no uso de propriedades para encapsulamento.

POO COM C#

1. Permite melhor organização, reutilização de código e manutenção facilitada, promovendo sistemas altamente escaláveis e flexíveis.

2. Criamos classes como moldes para objetos que possuem atributos e métodos, ou seja, características e comportamentos

PILARES DA POO

1. Abstração
2. Herança
3. Encapsulamento
4. Polimorfismo

ASSOCIAÇÕES

RELACIONAMENTOS

1. **Agregação**

2. **Composição**

Relacionamentos entre Classes

AGREGAÇÃO

Agregação (Relacionamento "Tem"):

- É um tipo de Composição mais fraco.
- Representa uma relação "todo-parte" onde as partes podem existir independentemente do todo.
- Exemplo: Um Carro agrega um Motor. O motor pode ser removido e usado em outro carro.

COMPOSIÇÃO

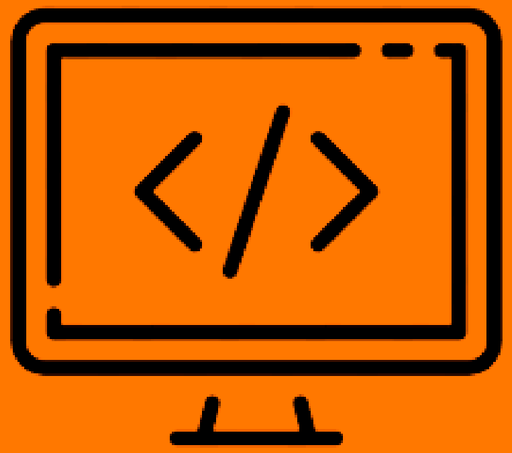
Composição (Forte): A parte não pode existir sem o todo.

- Exemplo: Um Carro compõe um Chassi. Se o carro for destruído, o Chassi não sobrevive sozinho.

CONSTRUTORES E COLECTIONS

1. Construtores:
   - Método automático que inicializa o objeto.
   - Garante que o objeto nasça em um estado válido.
   - Chamado usando new.

2. Coleções (Listas e Vetores):
   - Usadas para armazenar vários objetos.
   - List<T>: Coleção dinâmica (tamanho variável).
   - Polimorfismo: Permite armazenar objetos de diferentes classes filhas em uma lista do tipo da classe mãe (List<Pai>).



KEEP CALM

AND

PROGRAME ORIENTADO A OBJETOS

EXERCÍCIO 1

Cenário: Em Senhor dos Anéis, Durante a defesa de Gondor, cada combatente aliado possui um nome, uma linhagem e uma patente militar.

Tarefa:

1. Crie a classe CombatenteAliado.
2. Defina atributos para o nome, a linhagem (raça) e a patente (função) do combatente. (Todos devem ser privados).
3. Crie um Construtor que obrigue a inicialização desses três atributos ao criar um novo combatente.
4. Crie um método ApresentarUnidade() que imprima as informações completas do personagem.
5. Na Main, crie as instâncias de um Humano Capitão, depois Anão Guerreiro usando o construtor, por fim mais um à escolha.
6. Chame o método ApresentarUnidade() em todos os objetos.

EXERCÍCIO 2

Cenário: No mundo Pokémon, criaturas como o Arcanine representam o elemento Fogo, enquanto o Feraligatr representa o elemento Água. Cada espécie possui uma forma distinta de executar seus movimentos ofensivos.

Tarefa:

1. Crie uma classe base chamada Pokemon. Adicione um atributo público chamado Especie.
2. Crie um método virtual chamado ExecutarMovimento() na classe CriaturaPokemon que imprima uma mensagem genérica de ataque.
3. Crie as classes TipoFogo e TipoAgua que herdem da classe CriaturaPokemon.
4. Em cada classe filha, utilize o override no método ExecutarMovimento() para refletir o elemento específico (ex: "lança uma rajada de chamas" ou "dispara uma bolha de água").
5. No método Main, crie uma lista de objetos baseada na classe mãe (List<CriaturaPokemon>).
6. Instancie um Scorbunny (TipoFogo) e um Greninja (TipoAgua), adicione-os à lista e utilize um laço de repetição (foreach) para chamar o método ExecutarMovimento() de cada um, demonstrando o comportamento polimórfico.

EXERCÍCIO 3

Cenário:
A elfa Frieren carrega consigo conhecimentos e objetos de naturezas diferentes. O seu Baú de Relíquias é uma parte intrínseca de sua identidade e jornada (composição), enquanto o seu Kit de Acampamento é um conjunto de itens externos que ela utiliza para sobreviver na estrada, mas que existem independentemente dela (agregação).

Tarefa:

1. Crie a classe ReliquiaMagica com uma propriedade NomeItem.
2. Crie a classe BauDeReliquias que possui uma lista interna (List<ReliquiaMagica>). Esta classe deve ter um método chamado GuardarItem(). (Lembre-se: o Baú "possui" as relíquias).
3. Crie a classe ItemDeAcampamento (ex: "Saco de Dormir", "Pote de Cozinha").

EXERCÍCIO 3

4. Crie a classe Maga: Ela deve possuir nome, um atributo para o BauDeReliquias que é instanciado dentro da própria classe ou de seu construtor (Composição). Ela deve possuir uma lista de ItemDeAcampamento que é recebida como parâmetro no construtor ou via propriedade (Agregação).
5. Na Main:
    - Instancie uma lista de ItemDeAcampamento com pelo menos dois itens.
    - Instancie a Maga Frieren, passando a lista de itens de acampamento criada anteriormente.
    - Utilize o método da Maga (ou de seu Baú) para adicionar duas novas ReliquiaMagica (ex: "Anel de Prata", "Grimório Antigo").
    - Imprima no console a lista de todos os itens de acampamento que a Maga está utilizando no momento.

EXERCÍCIO 4

Cenário: Nas profundezas da Biblioteca da Universidade Miskatonic, um pesquisador encontra relatos de horrores indescritíveis. Embora cada horror venha de uma dimensão diferente, como o Shoggoth (uma massa amorfa de olhos e bocas) e a Morte Alada (uma criatura horrenda que cruza os céus), todos são catalogados como AbominacaoCosmica. Cada uma dessas entidades possui uma forma única de se manifestar no nosso plano.

Tarefa:

1. Crie uma classe base abstrata chamada AbominacaoCosmica:
   - Adicione um atributo para o Nome.
   - Crie um método abstrato (ou virtual) chamado Manifestar().
2. Crie a classe Shoggoth que herda de AbominacaoCosmica:
   - Sobrescreva o método Manifestar() para indicar um movimento visceral e caótico (ex: "A massa de protoplasma se arrasta borbulhando, moldando órgãos e olhos conforme avança").

EXERCÍCIO 4

- Crie a classe CacadorAlado que herda de AbominacaoCosmica:
  - Sobrescreva o método Manifestar() para indicar um movimento aéreo e perturbador (ex: "A criatura bate suas asas membranosas, mergulhando do vácuo estelar em um voo silencioso").
- No método main:
  - Crie uma lista ou vetor de AbominacaoCosmica.
  - Instancie um Shoggoth e um CacadorAlado, adicione-os à lista e percorra-a chamando o método Manifestar() para cada um.

FIM DA AULA!

zSh3rl0cK/**S01-monitoria**

1 Contributor | 0 Issues | 0 Stars | 0 Forks

zSh3rl0cK/S01-monitoria

Contribute to zSh3rl0cK/S01-monitoria development by creating an account on GitHub.

GitHub